A4 – Genetic Algorithms

Group number	Members
3	Mireia Calviño, Alan Fijal, Anna Scotti, and Anna Ymbern

1. Code explanation

```python
# Function 1: Download MNIST images and save them locally
def download_mnist_images():
    # Download MNIST dataset and store images and labels
    # Save the first 500 images and their labels to a folder

    mnist = fetch_openml('mnist_784', version=1, cache=True, as_frame=False) #fetch the MNIST dataset

    selected_images = mnist.data[:500] #select the first 500 images
    selected_labels = mnist.target[:500] #select the first 500 labels
    if not os.path.exists("mnist_images"): #check if the folder exists
        os.makedirs('mnist_images') #create a folder to save the images if the folder does not exist

    for i, (image, label) in enumerate(zip(selected_images, selected_labels)): #loop through the selected images and labels
        img_array = image.reshape(28, 28) #reshape the image to 28x28
        img = Image.fromarray(img_array.astype(np.uint8)) #convert the image to a PIL image
        file_name = f"mnist_images/img_{i}_label{label}.png" #create a file name for the image
        img.save(file_name) #save the image to the folder

    logger.success("MNIST images saved to 'mnist_images' folder.") #log the success of the function
    return selected_images, selected_labels #return the selected images and labels
```

The **download_mnist_images** function downloads the MNIST dataset and saves the first 500 digit images locally as PNG files. It requires no arguments as inputs because it always downloads the same fixed dataset and will process the first 500 images. Internally, it uses fetch_openml to access this dataset, and then reshapes each image from a 784-length vector to a 28x28 grayscale array. It checks for the folder mnist_images, creates if necessary, and saves each image using PIL with filenames and indicating their index and respective label. The images are then stored in a black and white format, keeping its pixel intensity. The function returns the image data and corresponding labels as NumPy arrays. These outputs are used in later functions for the analysis, cropping and classification. A success message is logged to confirm the completition.

```python
# Function 2: Analyze the images
def analyze_images(images):
    # Print image details: number, dimensions, pixel count, and range
    # Check if image is PNG (alpha channel)

    for i, image_data in enumerate(images): #loop through the images
        print(f"\n ---Image{i} ---") #print the image number
        print(f"Dimensions: {image_data.shape}") #print the dimensions of the image
        print(f"Pixel_count: {image_data.size} ") #print the pixel count of the image
        print(f"Range: ({image_data.min()}, {image_data.max()})") #print the range of the image

        has_alpha = False #initialise the has_alpha variable
        img_mode = None #initialise the img_mode variable

        if image_data.ndim == 3 and image_data.shape[-1] == 4: #check if the image has an alpha channel
            has_alpha = True  #set the has_alpha variable to True
            img_mode = "Implied RGBA (from shape)" #set the img_mode variable to "Implied RGBA (from shape)"

        elif image_data.ndim == 3 and image_data.shape[-1] == 2: #check if the image has an alpha channel
            has_alpha = True  #set the has_alpha variable to True
            img_mode = "Implied LA (from shape)" #set the img_mode variable to "Implied LA (from shape)"

        if img_mode: #if the img_mode variable is not None
            print(f"Interpreted Mode: {img_mode}") #print the img_mode variable

        print(f"Has Alpha Channel (Proxy for PNG Transparency): {has_alpha}") #print the has_alpha variable
```

The **analyze_images** function analyses an array of image data, printing details about each image's structure. It takes a list of NumPy arrays as input, representing grayscale images. For each image, it prints the index, shape, pixel count and the intensity range. It also checks the number of dimensions in order to

check if the image has an alpha (transparency) channel, which is found in PNG formats. If the image uses RGBA or LA format, it will print the interpreted mode. While this function doesn't modify or return anything, it helps to verify the image format and content before running other steps like cropping or histogram comparision.

```python
# Function 3: Crop the image identifying empty rows and columns
def crop_image(image):
    # Identify non-zero rows and columns
    # Crop the image to the non-zero region and pad to 28x28 if needed

    non_empty_rows = np.any(image > 0, axis=1) #identify the non-zero rows
    non_empty_columns = np.any(image > 0, axis=0) #identify the non-zero columns

    row_indices = np.where(non_empty_rows)[0] #identify the indices of the non-zero rows
    columns_indices = np.where(non_empty_columns)[0] #identify the indices of the non-zero columns

    if len(row_indices) == 0 or len(columns_indices) == 0: #if the row_indices or columns_indices are empty
        return image #return the image

    top, bottom = row_indices[0], row_indices[-1] + 1 #identify the top and bottom of the image
    left, right = columns_indices[0], columns_indices[-1] + 1 #identify the left and right of the image

    cropped = image[top:bottom, left:right] #crop the image to the non-zero region

    return cropped #return the cropped image
```

The **crop_image** function trims the unnecessary whitespace from a digit image by removing the rows and columns containing only zeros. It a 28x28 grayscale NumPy array as input and identifies the top, bottom, left and right edges of the region where the digit is drawn. Then, it crops the image to keep only this part, centering the digit and removing the black areas in its sorroundings. If the image is completely blank, it returns it unchanged. This helps prepare a leaner and more focused digit images for the analysis and classification.

```python
# Function 4: Plot histogram comparison
def plot_histogram_comparison(images, labels, max_labels_per_figure=5):
    # Select first occurrences of each label and compute histograms
    # Plot histograms in multiple figures if needed

    if images.size == 0 or labels.size == 0: #if the images or labels are empty
        logger.warning("Empty images or labels provided for histogram comparisons.") #log the warning
        return  #return None

    first_occurence_indices = {} #initialise the first_occurence_indices dictionary, that will store the first occurence of each label
    unique_labels_found = [] #initialise the unique_labels_found list, that will store the unique labels

    for index, label in enumerate(labels): #loop through the labels
        label_str = str(label) #convert the label to a string to assure the consistency of the keys
        if label_str not in first_occurence_indices: #if the label is not in the first_occurence_indices dictionary
            first_occurence_indices[label_str] = index #add the label to the first_occurence_indices dictionary
            unique_labels_found.append(label_str) #add the label to the unique_labels_found list

    try:
        sorted_unique_labels = sorted(unique_labels_found, key=lambda x: int(x)) #sort the unique labels numerically
    except ValueError: #if the labels could not be sorted numerically
        logger.warning("Labels could not be sorted numerically, using string sort.") #log the warning
        sorted_unique_labels = sorted(unique_labels_found) #sort the unique labels stringly

    if not sorted_unique_labels: #if the unique labels are not found
        logger.warning("No unique labels found.") #log the warning

    logger.success(f"Found unique labels: {', '.join(sorted_unique_labels)}") #log the success of the function

    num_labels = len(sorted_unique_labels) #identify the number of labels
    num_figures = math.ceil(num_labels / max_labels_per_figure) #identify the number of figures to plot the histograms
    logger.info(f"Plotting {num_labels} histograms across {num_figures} figure(s)") #log the number of histograms and figures

    epsilon = np.exp(-20) #Set epsilon to a small value to avoid log(0)
    for fig_idx in range(num_figures): #loop through the figures

        start_index = fig_idx * max_labels_per_figure #identify the start index of the figure
        end_index = min(start_index + max_labels_per_figure, num_labels) #identify the end index of the figure
        num_plots_on_this_figure = end_index - start_index #identify the number of plots on this figure

        if num_plots_on_this_figure <= 0: #if the number of plots on this figure is less than or equal to 0
            continue #continue to the next figure

        if num_plots_on_this_figure == 1: #if the number of plots on this figure is 1
            cols = 1 #identify the number of columns
            rows = 1 #identify the number of rows
        elif num_plots_on_this_figure <= 4: #if the number of plots on this figure is less than or equal to 4
            cols = 2 #identify the number of columns
            rows = math.ceil(num_plots_on_this_figure / 2) #identify the number of rows
        else: #if the number of plots on this figure is greater than 4
            cols = 3 #identify the number of columns
            rows = math.ceil(num_plots_on_this_figure / cols) #identify the number of rows

        _, axes = plt.subplots(rows, cols, figsize=(5 * cols, 4 * rows), squeeze=False) #create the figure and axes
        plot_counter = 0 #initialise the plot counter
        axes_flat = axes.flatten() #flatten the axes

        for i in range(start_index, end_index): #loop through the plots
            label = sorted_unique_labels[i] #identify the label
            image_index = first_occurence_indices[label] #identify the image index

            image_data = images[image_index] #identify the image data
            image_flat = image_data.flatten() #flatten the image data

            total_pixels = image_flat.size #identify the total number of pixels
            if total_pixels == 0: #if the total number of pixels is 0
                logger.warning(f"Image: {image_index} for label '{label}' has zero pixels. Skipping histogram.") #log the warning

            hist, bin_edges = np.histogram(image_flat, bins=50, range=(0, 256)) #create the histogram

            log_hist = np.log(hist.astype(np.float64) + epsilon) #identify the log of the histogram

            ax = axes_flat[plot_counter] #identify the axis
            ax.plot(bin_edges[:-1], log_hist, label=f"Digit '{label}'", color='darkturquoise', linewidth=1.0) #plot the histogram
            ax.set_title(f"Log-Normalised Histogram for Label {label}") #set the title of the histogram
            ax.set_xlabel("Pixel Intensity") #set the x label of the histogram
            ax.set_ylabel("Log of frequency") #set the y label of the histogram
            ax.grid(True, linestyle='--', alpha = 0.6) #set the grid of the histogram

            plot_counter += 1 #increment the plot counter


        for k in range(plot_counter, len(axes_flat)): #loop through the axes
            axes_flat[k].axis('off') #turn off the axis

        plt.tight_layout(rect=[0, 0.03, 1, 0.97]) #tight layout the plot


    plt.show() #show the plot
```

The **plot_histogram_comparison** function starts by checking whether the provided images or labels arrays are empty. If so, it logs a warning and exits immediately to avoid drawing blank plots. After, it initializes two containers; a dictionary to map each digit's string label to the index of its first occurrence, and a list to preserve that insertion order. As it iterates through labels, the first time it records a new digit, gets both the index and the label string. Once one representative index per digit is collected, it sorts those label strings by converting them to integers ("2" correctly precedes "10"). If any label can't be parsed as an integer, the code catches the resulting ValueError, logs a warning, and falls back to a simple string-based sort. With the sorted labels in hand, it calculates how many figures will be needed by dividing the total number of digits by **max_labels_per_figure,** (by default five) and rounding up.

For each figure "page," it determines the grid layout: a single cell when there's just one plot, a two-by-two grid for up to four, or a three-column arrangement when there are more. It calls plt.subplots(rows, cols, figsize=(5*cols, 4*rows), squeeze=False) and immediately flattens the axes array so each subplot can be addressed by a simple index.

Inside the grid, for each digit it pulls the stored image by its index and flattens the 28x28 array into a 1D vector of 784 pixels. Then it calls np.histogram to get the raw counts in 50 intensity bins. After, it converts the counts to float and replaces any zero-count bins with a extremely small number (epsilon = np.exp(-20)), to avoid any errors with log(0). With this it computes the log of the histogram with np.log(hist + epsilon) and it plots it vs intensity.

WEEK 2

```python
def classify_images_with_kdtree(images, labels):

    # Build KDTree
    if images.ndim > 2: #if the images are not flattened
        flattened_images = np.array([img.flatten() for img in images]) #flatten the images
    else:
        flattened_images = images #if the images are flattened, assign the images to flattened_images

    if not flattened_images.size: #if the flattened_images are empty
        logger.warning("No images to classify with improved KDTree.") #log the warning
        return np.array([]) #return an empty array

    kdtree = KDTree(flattened_images) #create a KDTree
    # Define a classification rule based on distances (ignore self-distance)
    distance, _ = kdtree.query(flattened_images, k=2) #query the KDTree
    nearest_distances = distance[:, 1] #identify the nearest distances, while ignoring the self-distance
    # Thresholding based on median distance
    median_distance = np.median(nearest_distances) #identify the median distance
    classification = np.zeros(len(images), dtype=int) #initialise the classification array
    classification[nearest_distances < median_distance] = 1 #set the classification to 1 if the nearest distance is less than the median distance

    logger.info("Finished KDTree classification") #log the success of the function
    return classification #return the classification
```

The **classify_images_with_kdtree** function builds a simple nearest-neighbour classifier over the MNIST images using a KD-Tree and then binarises each image based on how "close" it is to its neighbours. It requires the two inputs: image (which may be 28x28 arrays or already flattened) and labels (unused here except for length), and it returns a 1D NumPy array of 0s and 1s. Internally, it will first ensure that every image is a 1D vector, then constructs a KDTree on the full set. It calls query(flattened_images, k=2) to get each image's two nearest distances (ignoring the self-distance) and it extracts the second column as the nearest neighbor distance. After computing the median of the distances to serve as a threshold, it logs the threshold, then sets classification to 1 for any image whose nearest distance is lower or equal to the median and 0 otherwise. It returns a binary vector that feeds directly into the **genetic_algorithm** as one possible feature input.

```python
def classify_images_with_kdtree_improved(images, labels):
    """
    Improvement:
    - Continous score instead of binary
    - Clasifier only compares images of the same class
    """

    # Build KDTree
    if images.ndim > 2: #if the images are not flattened
        flattened_images = np.array([img.flatten() for img in images]) #flatten the images
    else:
        flattened_images = images #if the images are flattened, assign the images to flattened_images

    if not flattened_images.size: #if the flattened_images are empty
        logger.warning("No images to classify with improved KDTree.") #log the warning
        return np.array([]) #return an empty array

    n = len(flattened_images) #identify the number of images
    classification = np.zeros(n, dtype=float) #initialise the classification array

    for cls in np.unique(labels): #loop through the unique labels
        idx = np.where(labels == cls)[0] #identify the indices of the unique labels
        if len(idx) < 2: #if the number of indices is less than 2
            classification[idx] = 1.0 #set the classification to 1.0 as there is no neighbour to compare against
            continue #continue to the next label

        data = flattened_images[idx] #identify the data
        kdtree = KDTree(data) #create a KDTree
        distances, _ = kdtree.query(data, k=2) #query the KDTree
        nearest_distance = distances[:, 1] #identify the nearest distance, while ignoring the self-distance

        d_min, d_max = nearest_distance.min(), nearest_distance.max() #identify the minimum and maximum of the nearest distance
        if d_max - d_min < 1e-12: #if the maximum and minimum of the nearest distance are the same
            similarity = np.ones_like(nearest_distance) #set the similarity to 1
        else: #if the maximum and minimum of the nearest distance are not the same
            similarity = 1.0 - (nearest_distance - d_min) / (d_max - d_min) #set the similarity to the continous score

        classification[idx] = similarity #set the classification to the similarity

    logger.success("Finished continous same-class KDTree classification") #log the success of the function
    return classification #return the classification
```

The **classify_images_with_kdtree_improved** function builds on a basic KD-Tree classifier by assigning each image a continuous similarity score between 0 and 1, comparing only against peers of the same digit. It takes two inputs—images, which may be either flattened vectors or reshapeable 28×28 arrays, and labels, the corresponding digit classes—and returns a 1D NumPy array of float scores. Internally, it first flattens any 28×28 inputs into 1D vectors, logs a warning and returns an empty array if there's no data, then allocates a zeroed output array. It then loops over each unique label: if a class has only one example, that image is given a score of 1.0; otherwise it builds a KD-Tree on that class's subset, queries the two nearest neighbours for each point (skipping the self-match), and records the nearest-neighbour distances. After finding the minimum and maximum distances within the class, it checks if they're effectively identical—in which case every image scores 1.0—otherwise it linearly rescales each distance as *1.0 – (distance – min_distance) / (max_distance – min_distance)* so that the closest pairs approach 1 and the farthest approach 0. Those per-class similarity values are placed back into the full output array at their original indices, a success message is logged, and the continuous similarity array is returned for the use in the **genetic_algorithm**.

```python
def create_random_individual(images, individual_length, labels, kdtree_vector):
    # Select random indexes
    indexes = [] #initialise the indexes array
    for _ in range(individual_length): #loop through the individual length
        random_index = np.random.randint(0, len(images) - 1) #select a random index
        indexes.append(random_index) #append the random index to the indexes array

    return images[indexes], labels[indexes], kdtree_vector[indexes] #return the images, labels and kdtree vector
```

create_random_individual function generates a candidate for the genetic algorithm by randomly sampling a fixed number of images (with replacement) along with their labels and KD-Tree scores. It takes four inputs: images, individual_length, labels, and kdtree_vector and returns three arrays of length individual_length.

It builds a list of individual_length random indices in [0, len(images)-1] and then uses those indices to slice

and return the corresponding images, labels, and KD-Tree values. These three values are used for the initial GA population in **genetic_algorithm**.

```
def selection(population, fitness_values):
    # Select the top 2 individuals based on fitness

    parent_1 = population[fitness_values[0]] #identify the first parent
    parent_2 = population[fitness_values[1]] #identify the second parent

    return (parent_1, parent_2) #return the parents
```

The **selection** function picks the two fittest candidates from a given population. It takes in population (an array of individuals) and fitness_values (an array where the first two entries are the indices of the highest-fitness individuals), then simply returns the two population entries at those indices as a (parent_1, parent_2) tuple. These parents will be used for **crossover** and **mutation** in the next GA generation.

```
def crossover(parent1, parent2, individual_length):

    # Split parents into images, labels and kdtree
    parent1_images = parent1[0] #identify the first parent's images
    parent1_labels = parent1[1] #identify the first parent's labels
    parent1_kdtree = parent1[2] #identify the first parent's kdtree

    parent2_images = parent2[0] #identify the second parent's images
    parent2_labels = parent2[1] #identify the second parent's labels
    parent2_kdtree = parent2[2] #identify the second parent's kdtree

    # Perform crossover separately for images, labels and kdtree
    random_index = np.random.randint(0, individual_length - 1) #select a random index
    child_images = np.concatenate((parent1_images[:random_index], parent2_images[random_index:])) #concatenate the first parent's images and the second parent's
    child_labels = np.concatenate((parent1_labels[:random_index], parent2_labels[random_index:])) #concatenate the first parent's labels and the second parent's
    child_kdtree = np.concatenate((parent1_kdtree[:random_index], parent2_kdtree[random_index:])) #concatenate the first parent's kdtree and the second parent's

    return child_images, child_labels, child_kdtree # Return child as a tuple
```

The **crossover** function creates a new offspring by splicing together two parent individuals at a randomly chosen cut point. It takes three inputs: parent1 and parent2 (each a tuple of images, labels, and KD-Tree scores) and individual_length and returns a tuple of three arrays for the child. The function unpacks each parent into its images, labels, and scores. Then draws a random index in [0, individual_length-1] and builds the child's images by concatenating parent1_images[:cut] with parent2_images[cut:]. The same is done to the labels and KD-Tree arrays. The result is a new individual mixing the first segment from one parent with the second segment from the other. This tuple feeds directly into the **mutation** function.

```
def mutation(individual, individual_length, images, labels, kdtree_vector):

    individual_image, individual_label, individual_kdtree = individual #identify the individual's images, labels and kdtree
    # Select a mutation point randomly
    mutation_point = np.random.randint(0, individual_length) #select a mutation point

    # Select a new random image and corresponding label
    random_index = np.random.randint(0, len(images) - 1) #select a random index
    random_image = images[random_index] #identify the random image
    random_label = labels[random_index] #identify the random label

    new_images = individual_image.copy() #copy the individual's images
    new_labels = individual_label.copy() #copy the individual's labels
    new_kdtree = individual_kdtree.copy() #copy the individual's kdtree
    # Calculate new kdtree
    new_images[mutation_point] = random_image #replace the mutation point with the random image
    new_labels[mutation_point] = random_label #replace the mutation point with the random label
    new_kdtree[mutation_point] = kdtree_vector[random_index] #replace the mutation point with the random kdtree


    return new_images, new_labels, new_kdtree #New individual
```

The **mutation** function adds variation by swapping out a single "gene" in an existing individual for a randomly chosen sample from the full dataset. It takes as inputs an individual tuple (its images array, labels

array, and KD-Tree scores), the individual_length (number of genes), and the original master arrays images, labels, and kdtree_vector.
First it unpacks the three components of the individual. A random position ”mutation_point” is chosen anywhere from 0 up to (but not including) individual_length. Next, it picks a fresh random_index from the range of valid dataset indices. Copies of the individual’s image, label, and score arrays are then made so that the original is preserved. At the mutation point, the function replaces the old gene with the image and label at random_index, and updates the KD-Tree score from kdtree_vector[random_index]. The result is a new triplet of arrays (images, labels, and scores) where exactly one slot has been replaced by a random example from the dataset. This triplet will introduce diversity into the GA implementation in **genetic_algorithm** function.

```
def fitness(individual, pi_digits):
    # Calculate total fitness_value based on pi_quality (total correct matches with the image labels) + kdtree_quality (sum of kdtree_vector)
    _, individual_label, individual_kdtree = individual #identify the individual's labels and kdtree
    pi_quality = 0 #initialise the pi quality

    for i in range(len(individual_label)): #loop through the individual's labels
        if str(individual_label[i]) == str(pi_digits[i]): #if the individual's label is equal to the pi digits
            pi_quality += 1 #increment the pi quality

    kdtree_quality = np.sum(individual_kdtree) #identify the kdtree quality

    fitness_value = pi_quality + kdtree_quality #identify the fitness value



    return fitness_value, pi_quality, kdtree_quality #return the fitness value, pi quality and kdtree quality
```

The **fitness** function scores a candidate individual by combining how many of its labels match a target sequence (pi_digits) with the total of its KD-Tree similarity scores. It takes two inputs: the individual tuple (where the second element is its labels array and the third element is its KD-Tree scores array) and pi_digits, a sequence of the same length containing the desired digit values, and calculates a single numeric fitness value.
It begins by pulling out the individual’s labels and its KD-Tree vector. It then walks through each position in those labels, comparing the string form of the individual’s label to the corresponding entry in pi_digits. Every time they match, it adds one to a running “pi_quality” count. Once that loop finishes, it sums all entries in the KD-Tree scores array as “kdtree_quality.” The final fitness value is simply the sum of pi_quality and kdtree_quality, blending exact label matches with the continuous similarity measures into a single evaluation score. This values will determine the selection ranking in **genetic_algorithm** and feed into **plot_evolution**.

```
def fitness_improved(individual, pi_digits):
    """
    Improvement:
    - Weighted kdtree quality
    """
    # Calculate total fitness_value based on pi_quality (total correct matches with the image labels) + kdtree_quality (sum of kdtree_vector)
    _, individual_label, individual_kdtree = individual #identify the individual's labels and kdtree
    pi_quality = 0 #initialise the pi quality

    for i in range(len(individual_label)): #loop through the individual's labels
        if str(individual_label[i]) == str(pi_digits[i]): #if the individual's label is equal to the pi digits
            pi_quality += 1 #increment the pi quality

    kdtree_quality = np.sum(individual_kdtree) #identify the kdtree quality

    weight_kdtree = 0.5 #set the weight of the kdtree to 0.5
    fitness_value = pi_quality + weight_kdtree * kdtree_quality #identify the fitness value



    return fitness_value, pi_quality, kdtree_quality #return the fitness value, pi quality and kdtree quality
```

The **fitness_improved** function enhances the original fitness calculation by applying a weight to the KD-Tree similarity component. It takes an individual tuple (images, labels, KD-Tree scores) and a pi_digits sequence of the same length, and returns a three-element tuple: (fitness_value, pi_quality, kdtree_quality). It begins by extracting the individual’s labels and KD-Tree vector, then walks through each

position comparing the label to the corresponding pi_digits entry to count pi_quality (the number of exact matches). It sums the KD-Tree scores as kdtree_quality, multiplies that sum by a fixed weight of 0.5, and adds it to pi_quality to form the overall fitness_value. Finally, it returns the combined fitness_value along with the raw pi_quality and weighted kdtree_quality. This values will determine the selection ranking in **genetic_algorithm** and feed into **plot_evolution**.

```
def plot_evolution(best_pi_quality_values, avg_pi_quality_values, best_kdtree_values, avg_kdtree_values):

    _, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5)) #create the figure and axes

    generations = range(1, len(best_pi_quality_values) + 1) #identify the generations


    ax1.plot(generations, best_pi_quality_values, 'b-', linewidth=2, label='Best pi_quality') #plot the best pi quality
    ax1.plot(generations, avg_pi_quality_values, 'c--', linewidth=1.5, label='Avg pi_quality') #plot the average pi quality
    ax1.set_xlabel('Generation') #set the x label
    ax1.set_ylabel('Fitness (pi_quality)') #set the y label
    ax1.set_title('Fitness Evolution - pi_quality') #set the title
    ax1.grid(False) #turn off the grid
    ax1.legend() #show the legend
    if best_pi_quality_values: #if the best pi quality values are not empty
        ax1.set_ylim(min(0, np.min(avg_pi_quality_values)-1 if avg_pi_quality_values else 0),
                     max(10.5, np.max(best_pi_quality_values)+0.5 if best_pi_quality_values else 10.5)) #set the y limits


    ax2.plot(generations, best_kdtree_values, 'r-', linewidth=2, label='Best kdtree_vector') #plot the best kdtree vector
    ax2.plot(generations, avg_kdtree_values, color='orange', linestyle='--', linewidth=1.5, label='Avg kdtree_vector') #plot the average kdtree vector
    ax2.set_xlabel('Generation') #set the x label
    ax2.set_ylabel('Fitness (kdtree_vector)') #set the y label
    ax2.set_title('Fitness Evolution - kdtree_vector') #set the title
    ax2.grid(False) #turn off the grid
    ax2.legend() #show the legend
    if best_kdtree_values : #if the best kdtree values are not empty
        ax2.set_ylim(min(4.0, np.min(avg_kdtree_values)-0.5 if avg_kdtree_values else 4.0),
                     max(8.5, np.max(best_kdtree_values)+0.5 if best_kdtree_values else 8.5)) #set the y limits


    plt.tight_layout() #tight layout the plot
    plt.show() #show the plot
```

The **plot_evolution** function visualises how both the exact-match quality (pi_quality) and the continuous KD-Tree similarity scores evolve over successive generations of the genetic algorithm. It accepts four parallel sequences—best and average values for pi_quality, and best and average values for the KD-Tree vector—each indexed by generation number, and produces a single Matplotlib figure split into two side-by-side plots.
It begins by creating a 15×5-inch canvas with two axes laid out in one row. A generations range is then defined from 1 up to the number of recorded generations. On the left subplot it draws the best pi_quality as a solid blue line and the average as a dashed cyan line, labels the X-axis “Generation” and Y-axis “Fitness (pi_quality),” gives the plot a descriptive title, hides the grid, and adds a legend. It then sets Y-limits so that the curve never dips below zero and always leaves a small margin above the highest best-value, defaulting to [0, 10.5] if the data are empty. The right subplot mirrors this process for the KD-Tree scores—plotting the best as a solid red line and the average as a dashed orange line, relabeling the Y-axis “Fitness (kdtree_vector)” and using its own sensible default limits of [4.0, 8.5] when needed. Finally, plt.tight_layout() is called to prevent any overlapping elements, and plt.show() renders both plots at once. The function does not return any value; its job is purely to display the fitness-over-time curves.

```python
def plot_best_individual(best_individual, pi_digits):

    best_images, best_labels, _ = best_individual  #identify the best images, labels and kdtree
    pi_digits_for_fitness = [str(p) for p in pi_digits] #identify the pi digits for fitness

    total_fitness_val, pi_quality_val, kdtree_quality_val = fitness(best_individual, pi_digits_for_fitness) #identify the total fitness value, pi quality value and

    num_images = len(best_images) #identify the number of images

    if num_images != 10: #if the number of images is not 10
        print(f"Warning: Expected 10 images for 2x5 plot, got {num_images}. Output may differ from expected.") #print the warning

    cols = 5 #identify the number of columns
    rows = (num_images + cols -1) // cols if num_images > 0 else 1 #identify the number of rows

    fig, axes = plt.subplots(rows, cols, figsize=(12, 5)) #create the figure and axes
    if num_images == 0: #if the number of images is 0
        axes = np.array([]) #set the axes to an empty array
    elif num_images == 1 and rows == 1 and cols == 1: #if the number of images is 1, the number of rows is 1 and the number of columns is 1
        axes = np.array([axes]) #set the axes to an array
    axes = axes.flatten() #flatten the axes

    title_str = f"Best-Fit Individual Images (Fitness value: {total_fitness_val:.0f}, Pi quality: {pi_quality_val:.0f}, KDTree quality: {kdtree_quality_val:.0f})"
    fig.suptitle(title_str, fontsize=12) #set the title of the figure

    for i in range(num_images): #loop through the images
        img = best_images[i] #identify the image
        if img.ndim == 1 and img.shape[0] == 28*28: #if the image is 1D and the shape of the image is 28x28
            img = img.reshape(28, 28) #reshape the image to 28x28

        axes[i].imshow(img, cmap='gray') #plot the image
        current_pi_digit = pi_digits[i] if i < len(pi_digits) else "N/A" #identify the current pi digit
        axes[i].set_title(f"Label: {best_labels[i]}\nReal: {current_pi_digit}", fontsize=9) #set the title of the image
        axes[i].axis('off') #turn off the axis


    for i in range(num_images, len(axes)): #loop through the images
        axes[i].axis('off') #turn off the axis

    plt.tight_layout(rect=[0, 0.03, 1, 0.95]) #tight layout the plot
    plt.show() #show the plot
```

The **plot_best_individual** function displays the champion candidate from the genetic algorithm alongside the target π-digit sequence and its fitness breakdown. It accepts two inputs—best_individual, a tuple of (images array, labels array, KD-Tree scores) for the top-scoring solution, and pi_digits, the desired sequence of digit values—and produces a Matplotlib figure of those images annotated with both their predicted label and the "real" π-digit.

First, it unpacks the individual's images and labels and calls the plain fitness routine (passing pi_digits as strings) to compute the total fitness, π-match count, and KD-Tree quality. It determines how many images must be shown and, if this isn't the expected ten, it emits a warning that the layout may differ. Columns are fixed at five; the number of rows is computed to fit all images. A figure with that many subplots is created and flattened for easy indexing. A super-title then displays the overall fitness, π-quality, and KD-Tree quality in one summary line.The function loops through each image: if it's still a 1D vector of length 784, it reshapes it to 28×28; it then renders it in grayscale, titles the subplot with both the candidate's label and the corresponding "real" π digit, and hides the axes. Any leftover subplot slots are also switched off. Finally, tight_layout is applied with a small top margin for the title, and plt.show() brings up the complete montage. No value is returned—its sole purpose is to give an at-a-glance visual of the best individual's performance.

```python
def genetic_algorithm(images, labels, kdtree_vector, improvements=False):

    # Step 1: Define the algorithm parameters (e.g., population size, number of generations, mutation rate)
    num_generations = 100  # Adjust as needed
    population_size = 50 # Adjust as needed
    individual_length = 10 #identify the individual length
    pi_digits = [1,4,1,5,9,2,6,5,3,5] #identify the pi digits
    mutation_rate = 0.2 #identify the mutation rate

    best_pi_quality_values = [] #initialise the best pi quality values
    avg_pi_quality_values = [] #initialise the average pi quality values
    best_kdtree_values = [] #initialise the best kdtree values
    avg_kdtree_values = [] #initialise the average kdtree values

    # Step 2: Initialize the population with randomly selected individuals
    current_population = [] #initialise the current population
    for _ in range(population_size): #loop through the population size
        random_individual = create_random_individual(images, individual_length, labels, kdtree_vector) #create a random individual
        current_population.append(random_individual) #append the random individual to the current population


    # Step 3: Evolve the population over multiple generations
    for _ in range(num_generations): #loop through the number of generations
        # Step 3.1: Compute the fitness score for each individual
        fitness_score = {} #initialise the fitness score
        pi_qualities = [] #initialise the pi qualities
        kdtree_qualities = [] #initialise the kdtree qualities

        for index, individual in enumerate(current_population): #loop through the current population
            if not improvements: #if the improvements are not used
                current_fitness_score, pi_quality, kdtree_quality = fitness(individual, pi_digits) #identify the current fitness score, pi quality and kdtree quality
            else:
                current_fitness_score, pi_quality, kdtree_quality = fitness_improved(individual, pi_digits)


            fitness_score[index] = current_fitness_score #assign the current fitness score to the fitness score
            pi_qualities.append(pi_quality) #append the pi quality to the pi qualities
            kdtree_qualities.append(kdtree_quality) #append the kdtree quality to the kdtree qualities

        sorted_indices = sorted(fitness_score.keys(), key=lambda x: fitness_score[x], reverse=True) #sort the indices of the fitness score

        best_pi_quality_values.append(pi_qualities[sorted_indices[0]]) #append the best pi quality to the best pi quality values
        avg_pi_quality_values.append(sum(pi_qualities) / len(pi_qualities)) #append the average pi quality to the average pi quality values
        best_kdtree_values.append(kdtree_qualities[sorted_indices[0]]) #append the best kdtree quality to the best kdtree values
        avg_kdtree_values.append(sum(kdtree_qualities) / len(kdtree_qualities)) #append the average kdtree quality to the average kdtree values

        # Step 3.2: Select the best individuals for reproduction
        best_individuals = selection(current_population, sorted_indices) #identify the best individuals for reproduction

        # Step 3.3: Generate new offspring using crossover and mutation
        new_population = [] #initialise the new population
        new_population.append(current_population[sorted_indices[0]]) #append the current population to the new population
        new_population.append(current_population[sorted_indices[1]]) #append the current population to the new population
        while len(new_population) < population_size: #loop through the population size
            new_offspring_crossed = crossover(best_individuals[0], best_individuals[1], individual_length) #identify the new offspring crossed
            random_prob = np.random.rand() #identify the random probability
            if random_prob < mutation_rate: #if the random probability is less than the mutation rate
                new_offspring_crossed_mutated = mutation(new_offspring_crossed, individual_length, images, labels, kdtree_vector) #identify the new offspring crossed mutated
                new_population.append(new_offspring_crossed_mutated) #append the new offspring crossed mutated to the new population
            else: #if the random probability is greater than the mutation rate
                new_population.append(new_offspring_crossed) #append the new offspring crossed to the new population



        # Step 3.4: Replace the old population with the newly created individuals
        current_population = new_population #replace the old population with the newly created individuals


    # Step 4: Retrieve the best-performing individual from the final generation
    final_fitness_scores = {} #initialise the final fitness scores
    for index, individual in enumerate(current_population): #loop through the current population
        current_fitness_score, pi_quality, kdtree_quality = fitness(individual, pi_digits) #identify the current fitness score, pi quality and kdtree quality
        final_fitness_scores[index] = current_fitness_score #assign the current fitness score to the final fitness scores

    sorted_indices = sorted(final_fitness_scores.keys(), key=lambda x: final_fitness_scores[x], reverse=True) #sort the indices of the final fitness scores

    best_performing_individual = current_population[sorted_indices[0]] #identify the best performing individual

    # Step 5: Plot the fitness evolution
    plot_evolution(best_pi_quality_values, avg_pi_quality_values, best_kdtree_values, avg_kdtree_values) #plot the fitness evolution
    # Step 6: Plot the image output of the best individual
    plot_best_individual(best_performing_individual, pi_digits) #plot the image output of the best individual
```

The **genetic_algorithm** function orchestrates the entire evolutionary search over your MNIST-based individuals, blending exact-digit matches and KD-Tree similarity into a candidate that best approximates a target π-digit sequence. It takes four arguments—images, labels, kdtree_vector, and an optional improvements flag to toggle the weighted fitness—and produces two visual outputs (the fitness-over-time curves and the best individual's image montage) without returning any value. First, it sets up its parameters: a fixed ten-gene individual length, fifty candidates per generation, one hundred generations, and a mutation rate of 20%. The π-digit target sequence is hard-coded as [1,4,1,5,9,2,6,5,3,5]. It then initializes a random starting population by repeatedly sampling ten images (with labels and KD-Tree scores) using create_random_individual. Across each generation, it evaluates every candidate's fitness—either with the basic or "improved" weighted routine—collecting both exact-match counts and KD-Tree sums. It records the best and average values for both metrics, then selects the two highest-scoring individuals via selection. Those two are carried over unchanged into the next generation, and the remainder of the new population is filled by

repeatedly crossing over the pair and, with 20% probability, mutating the offspring via mutation. Once the population is full, it replaces the old generation and repeats. After all generations have run, it re-evaluates the final population to find the single best performer, then calls plot_evolution to show how pi-match quality and KD-Tree similarity evolved over time, followed by plot_best_individual to display that champion's ten images alongside their predicted versus "real" π digits and overall fitness breakdown.

The main():

```python
# Main function to call the other functions
def main():
    ################### SESSION 1 ###################
    # Step 1: Download MNIST images
    # Call download_mnist_images() to download and save images
    images, labels = download_mnist_images()
    # Step 2: Analyze the first 5 images
    # Call analyze_images() for the first 5 images
    analyze_images(images[:5])
    # Step 3: Crop and resize the first 20 images
    # Call crop_image() for the first 20 images
    cropped_images = [] #keep track of the cropped images
    for i in range(20): #loop through the first 20 images
        img = images[i].reshape(28, 28) #reshape the image to 28x28
        cropped_img = crop_image(img) #crop the image
        cropped_images.append(cropped_img) #append the cropped image to the cropped_images list

    # Step 4: Display original vs cropped images (sample of 4)
    # Show comparison using matplotlib
    plt.figure(figsize=(10, 5)) #create the figure
    for i in range(4): #loop through the first 4 images
        # Original image
        plt.subplot(2, 4, i+1) #plot the original image
        plt.imshow(images[i].reshape(28, 28), cmap='gray') #plot the original image
        plt.title(f"Original {labels[i]}") #set the title of the original image
        plt.axis('off') #turn off the axis

        # Cropped image
        plt.subplot(2, 4, i+5) #plot the cropped image
        plt.imshow(cropped_images[i], cmap='gray') #plot the cropped image
        plt.title(f"Cropped {labels[i]}") #set the title of the cropped image
        plt.axis('off') #turn off the axis

    plt.tight_layout() #tight layout the plot
    plt.show() #show the plot

    # Step 5: Plot histogram comparison for cropped images
    # Call plot_histogram_comparison() for the cropped images
    plot_histogram_comparison(images, labels)


    ################### SESSION 2 ###################
    # Step 6: Generate KDTree vector
    kdtree_vector = classify_images_with_kdtree(images, labels) #generate the kdtree vector
    kdtree_vector_improved = classify_images_with_kdtree_improved(images, labels) #generate the kdtree vector with improvements

    # Step 7: Run GA
    genetic_algorithm(images, labels, kdtree_vector_improved, improvements=True) #run the genetic algorithm with improvements

    genetic_algorithm(images, labels, kdtree_vector) #run the genetic algorithm
```

The function orchestrates the entire two-stage workflow. In **Session 1**, it begins by calling download_mnist_images() to fetch and locally save the first 500 MNIST digits, then takes the first five and hands them to analyze_images() for a quick print-out of shapes and value ranges. It next reshapes the first twenty into 28×28 arrays, crops each via crop_image(), and stores the results. A 2×4 Matplotlib figure then juxtaposes the first four originals against their cropped counterparts, and finally plot_histogram_comparison(images, labels) is invoked to generate the log-normalised histograms for every digit class.

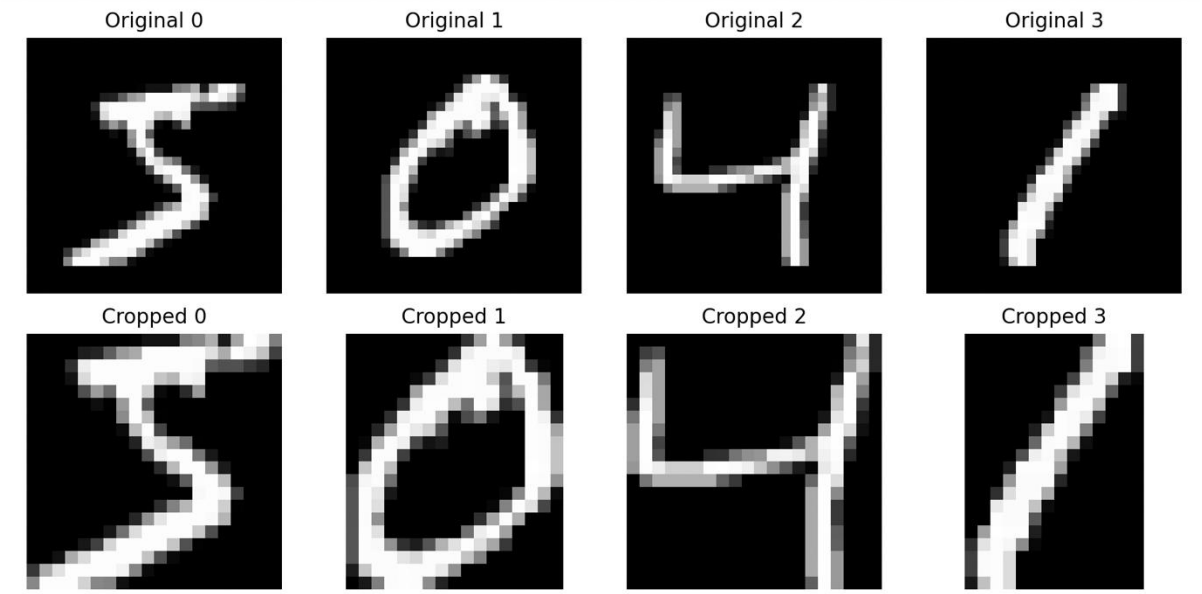
In **Session 2**, main computes two KD-Tree-based feature vectors—one basic, one "improved"—by calling classify_images_with_kdtree and classify_images_with_kdtree_improved. Passing the improved scores into genetic_algorithm(images, labels, kdtree_vector_improved, improvements=True) kicks off the

evolutionary search: it evolves a population of ten-digit individuals toward the hard-coded π-digit target, then produces two visual summaries (fitness over time and the best individual montage). The guarded if __name__ == "__main__": main() ensures this pipeline runs only when the script is executed directly.

Results obtained

WEEK 1

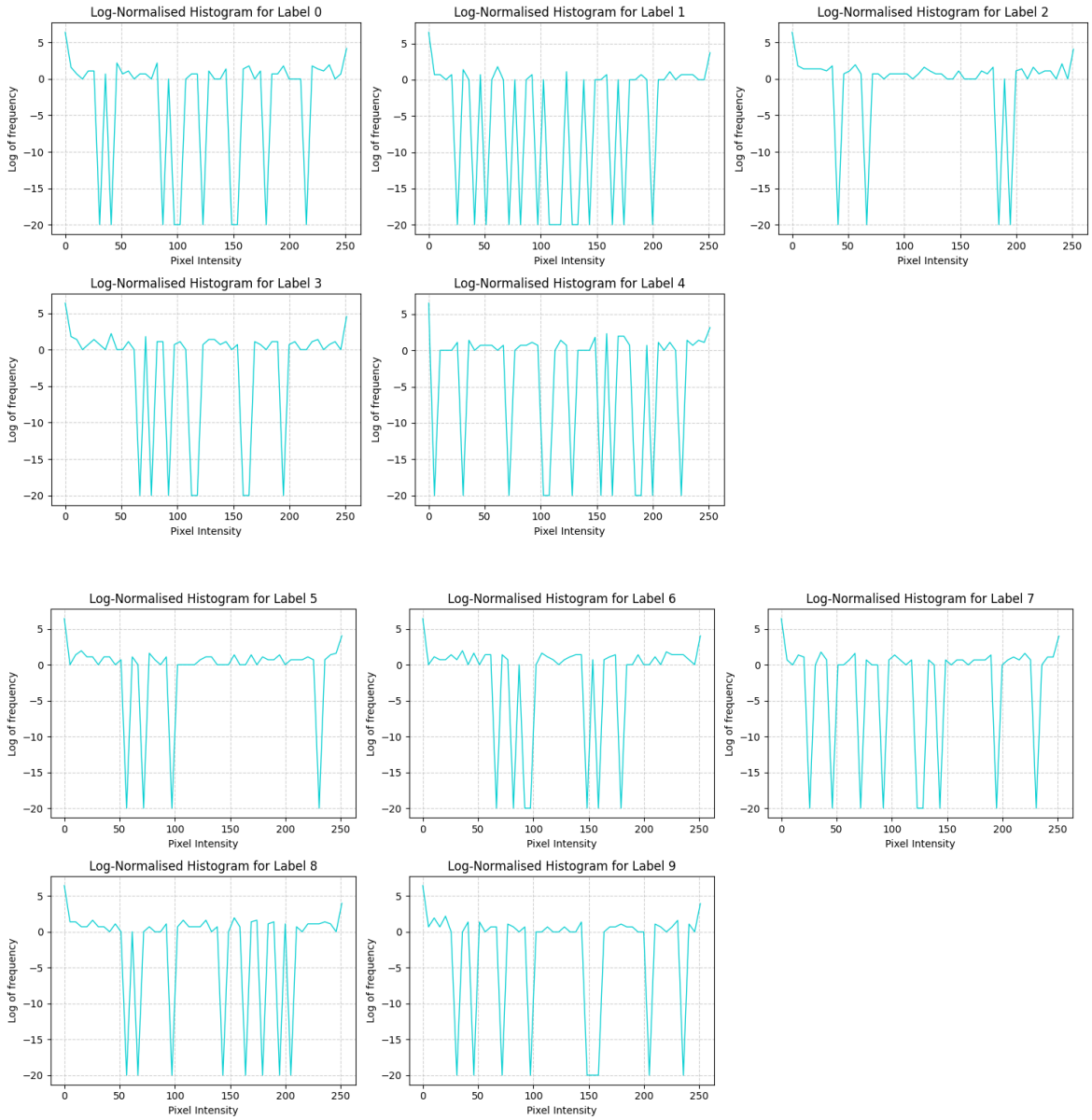
Original vs Cropped Digits



This image offers a side-by-side visual comparison between the original MNIST digit images and their cropped counterparts, helping to clearly illustrate the transformation that occurs during preprocessing. In the top row, we see the digits exactly as they are provided in the MNIST dataset: in a fixed 28x28 pixel grid, regardless of how much space the digit actually occupies. As a result, many of these images appear with large areas of black around the digit, which contributes nothing meaningful to the image, but only takes up space. Moreover, this superfluous border can make the digit appear smaller or off-center, potentially affecting both human interpretation and algorithmic processing.
The bottom row displays the same four digits after being processed with the crop_image function. This function scans each image to identify rows and columns to effectively eliminate unused space. What remains is a much more tightly bounded version of the digit, where the number is centered and fills the available space more efficiently. The cropped images retain all the relevant information, so the shape and stroke of the digit, while eliminating the unnecessary background. This not only improves visual clarity but can also enhance performance in tasks like digit classification, where reducing noise and focusing on key features is essential.

The comparison makes it easy to appreciate how a relatively simple preprocessing step like cropping can significantly clean up the data and help algorithms learn more effectively.

Log-Normalized Histograms of Pixel Intensities



This picture displays a collection of histogram plots that provide insight into the pixel intensity distributions of different handwritten digits from the MNIST dataset, specifically one for each digit from 0 to 9.

These histograms are not calculated from the entire dataset but are instead based on the first example of each digit that appears in the data. This approach helps to quickly capture the unique pixel characteristics of each digit without overwhelming the visualization with redundant data.

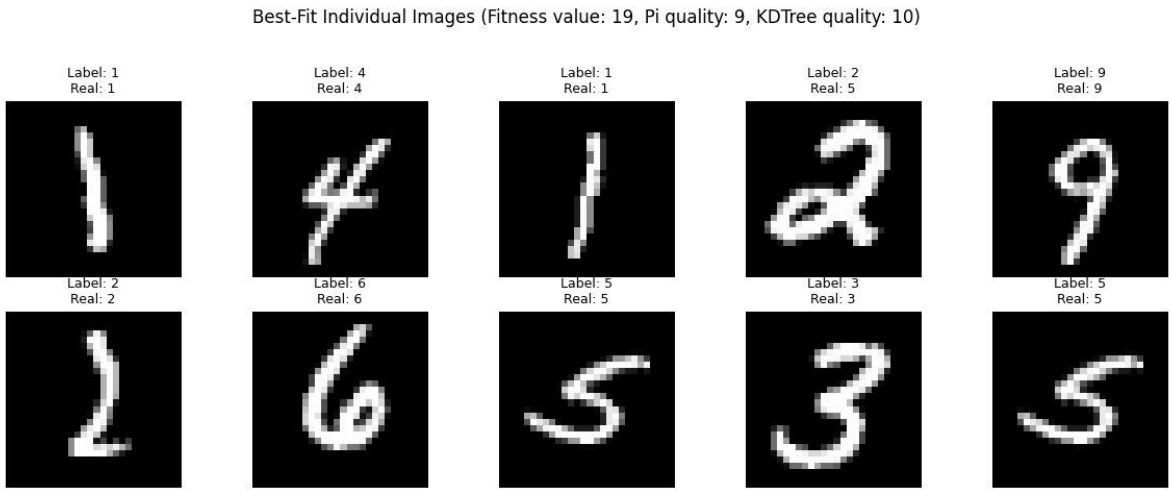
Each subplot represents a digit and plots pixel intensities along the X-axis, ranging from 0 (black) to 255 (white), while the Y-axis shows the logarithm of the raw count in each intensity bin. To avoid undefined values at zero counts, any bin with no pixels is replaced by a very small number (epsilon = 10^-20) before taking the logarithm, so you see very deep dips around -20 wherever no pixels fell. Peaks rise up to about 5-6 for the most common intensities (for example for dark background and the stroke gray levels). This makes both the prominent peaks and sparse gaps clearly visible.

The patterns of the spikes vary from one image to another, even with the same digit. This is because some samples are “cleaner” (as they have less gray levels) while other will be noisier (as they have more gray

levels). In this case, we can see that for example label 1 has a lot of spikes down to -20, meaning that there are a lot of intensities that are represented with zero pixels. This makes sense for 1 since it's a simple digit that tends to look "cleaner" (having less gray levels). On the other hand, we can see that for example label 5 or 2 doesn't have a lot of spikes, meaning there's consistency in the frequency of pixels across the different pixel intensities. This means that the digits may have more details, as different stroke widths or some background noise, which makes sense for digits like 5 or 2 as they have curves and are more complex.

These plots were generated using the plot_histogram_comparison function, and they visually demonstrate how each digit class has its own pixel "signature." This kind of analysis is valuable for understanding how the appearance of each digit translates into numeric data.

WEEK 2
Before improvements

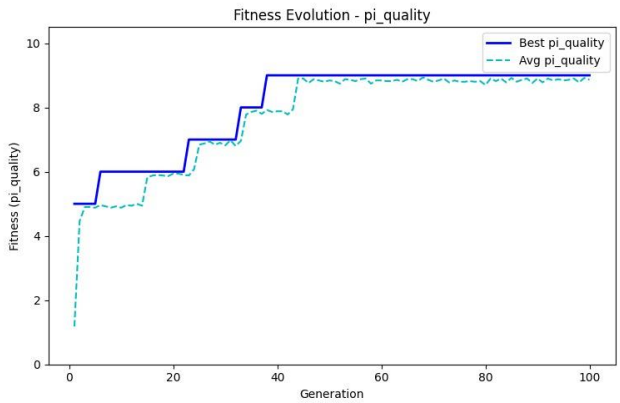


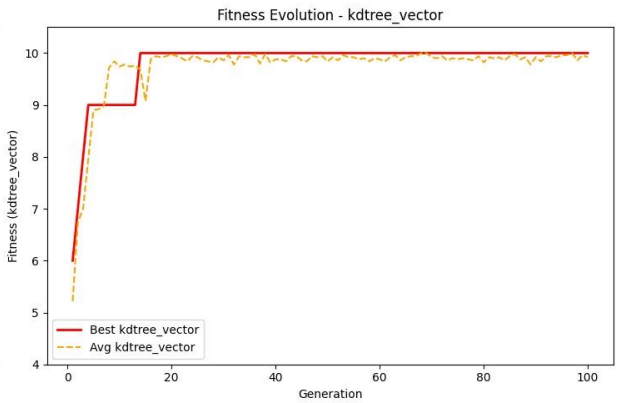
The image shows the result of running the genetic algorithm before any improvements were applied.

This initial version already produced an impressively accurate sequence, nearly replicating the target π-digit string [1, 4, 1, 5, 9, 2, 6, 5, 3, 5]. Out of the 10 digits, 9 were correctly matched, demonstrating the algorithm's strong capacity to identify correct digit labels.

Alongside this high accuracy, the individual also scored a perfect 10 in KD-Tree similarity, indicating that all selected digits were visually representative and clean examples from the dataset. The only mistake was choosing a '2' instead of a '5' at one position.

Overall, even without enhancements, the model performed remarkably well, showing a balanced focus on label correctness and image quality.





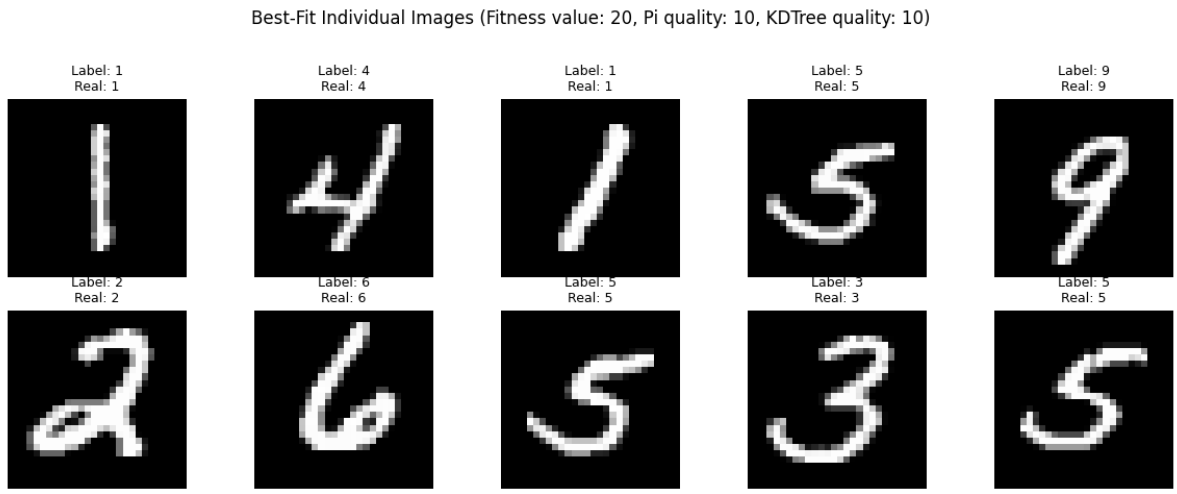
These two plots show how the genetic algorithm improved over 100 generations.

In the left plot, we see the number of correct digits matching the π sequence (pi_quality). It starts low but steadily climbs. The average score also rises, showing that the whole population improves over time, not just the top candidate.

The right plot shows the visual quality (kdtree_vector) based on how typical and clean the digits are. Here, we see very fast improvement: most individuals quickly settle on clean examples.

Overall, the algorithm quickly learns to pick clean digits and gradually gets better at matching the correct π sequence.

After improvements

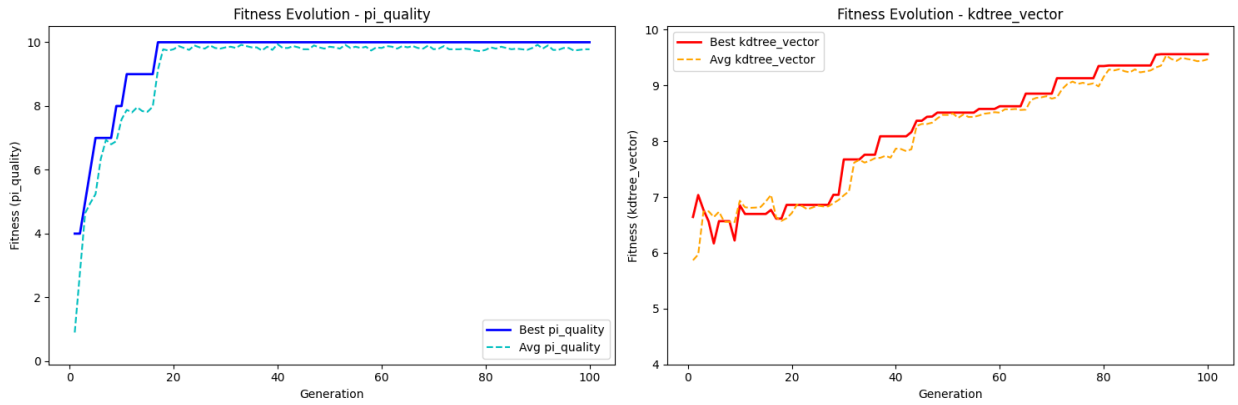


In this image, we see the effects of applying improvements to the KD-Tree similarity calculation and fitness evaluation.

Specifically, instead of using a basic nearest-neighbor distance, the improved classifier evaluates similarity only among images of the same digit class, resulting in more meaningful comparisons.

The genetic algorithm then uses this refined similarity in a weighted fitness function that still rewards matching the π-digit sequence, but also gives credit for choosing the most typical-looking digits. The visual results reflect this: the digits are not only mostly accurate but also appear better centered, more consistent, and clearer in stroke.

This refinement ensures that the best candidates aren't just correct in label, but also optimal in quality.



This image showcases another example of a strong result from the improved algorithm, highlighting its consistency and reliability.

It demonstrates that the evolved population doesn't just stumble upon one good match, but can repeatedly generate high-fitness individuals with different digit combinations that still align closely with the target π sequence.

This variation in solutions, while maintaining high pi-quality and strong KD-Tree similarity, shows that the improvements introduced in the second stage lead to a more stable and effective evolutionary process.

By selecting for both correctness and visual clarity, the algorithm reliably produces high-quality outputs even across different generations or random seeds.

Although the KD-Tree score is slightly higher before improvement, π quality converges much faster now: an exploratory strategy that helps us make steady progress on what really matters (correct labels), and avoiding getting stuck in local minima (close but wrong label)

2. Group management

The group worked very well together throughout the project, maintaining good communication and a balanced distribution of tasks. Everyone followed the instructions carefully and made thoughtful contributions, which helped us stay organized and on track. One of the main challenges we encountered was implementing the KD-Tree, as it was a new concept for most of us and initially felt a bit more complex than expected. However, by collaborating closely, discussing our doubts, and researching together, we were able to fully understand and implement it successfully. Overall, the group dynamic was positive, and the work was carried out efficiently and with strong teamwork.

3. Contributions of each member

Anna Y.
Anna collaborated on the codebase with her teammates, and her primary focus was on the written documentation. She did detailed explanations for every function, ensuring that the logic, inputs and outputs, and internal steps were presented with clarity.

Anna S.
Anna also contributed to the coding effort, then took charge of the results section in the document. She organized and interpreted the experimental outputs and wove those findings into a cohesive narrative that highlights our key observations.

Alan
Alan brought deep technical expertise to the project's implementation. He led the development of the KD-Tree and genetic-algorithm modules, optimized performance-critical sections, ensuring our algorithms ran correctly and efficiently.

Mireia
Mireia provided vital support on both fronts. She worked on the code closely, debugging issues, integrating new features, and helped polish the documentation along with the others.

4. Self-evaluation

	Evaluated			
Evaluator	Anna Ymbern	Anna Scotti	Alan Fijal	Mireia Calvino
Anna Ymbern	-	10/10	10/10	10/10
Anna Scotti	10 /10	-	10 /10	10 /10
Alan Fijal	10/10	10/10	-	10/10
Mireia Calvino	10/10	10/10	10/10	-